Investigation-First Development Protocol (IFDP)

Overview

The **Investigation-First Development Protocol (IFDP)** is a systematic, evidence-driven approach to diagnosing and fixing complex production issues. It prioritizes rigorous evidence gathering, structured observation, and phased implementation over reactive “quick fixes.”

This methodology has proven highly effective at preventing regressions while rapidly resolving critical issues. It is based on the principle that understanding the root cause thoroughly before implementing changes significantly reduces the risk of introducing new bugs or exacerbating existing problems.

The Three-Phase Pattern

Phase A: Investigation & Diagnosis

Goal: Understand the root cause with concrete evidence.

Golden Rule: NO code changes are permitted during this phase. This phase is purely observational and analytical.

Steps:

1. **Flow Mapping**
   - Trace the complete logic flow through the codebase
   - Document file paths and line numbers
   - Identify all branches and conditional logic
   - Map how data flows through the system

2. **Evidence Gathering**
   - Execute database queries to identify failure patterns
   - Analyze system logs and monitoring data
   - Compare metrics before and after the issue onset
   - Collect concrete data points that support or refute hypotheses

3. **Code Review**
   - Inspect relevant logic in affected modules
   - Check environment variables and configuration settings
   - Review error handling and edge cases
   - Look for inconsistencies or potential conflict points

4. **Hypothesis Formation**
   - State the root cause based on data, not intuition
   - Rank potential causes by likelihood and impact
   - Develop 3-5 alternative fix strategies
   - Document evidence supporting the primary hypothesis

Deliverables:

- Clear root cause hypothesis with supporting evidence
- Prioritized fix strategy (ranked by effectiveness and risk)
- A “STOP” point for user approval before proceeding to implementation
- Complete documentation of the investigation process

Phase A Completion Condition:

> “Phase [X]A investigation complete. Root cause: [Finding]. Awaiting approval to proceed to Phase [X]B implementation.”

Phase B: Implementation

Goal: Apply the targeted fix based on Phase A findings in a controlled, observable manner.

Steps:

1. **Surgical Fixes**
   - Implement only the approved fixes identified in Phase A
   - Avoid scope creep or opportunistic improvements
   - Make minimal, targeted changes to the codebase

2. **Observability First**
   - Add structured logging (checkpoints) BEFORE changing behavior
   - Log key decision points and state transitions
   - Include context (city, slot, provider, user ID as applicable)
   - Capture both success and error cases

3. **Testing**
   - Implement unit tests covering the fix and edge cases
   - Run manual generation tests (e.g., 10 successful runs required)
   - Verify backward compatibility with existing functionality
   - Document all test results and pass/fail criteria

4. **Deployment**
   - Deploy to staging/production in a controlled manner
   - Verify clean logs and expected observability signals
   - Monitor initial deployment for immediate issues
   - Prepare rollback plan if needed

Logging Standard

All logging events should follow this structure:

```
{
  "event_type": "fix_checkpoint",
  "context": {
    "city": "...",
    "slot": "...",
    "provider": "...",
    "user_id": "..." (if applicable)
  },
  "message": "...",
  "error_details": {
    "http_status": "...",
    "error_message": "...",
    "response_body_excerpt": "..." (first 500 chars)
  }
}
```

Deliverables:

- Before/after code comparison (diff)
- Complete test results with pass/fail counts
- Deployment confirmation with timestamp
- Observability checkpoint verification

Phase B Completion Condition:

> "Phase [X]B implementation complete. Deployed successfully. Ready for Phase [X]C production monitoring."

Phase C: Production Verification

Goal: Prove the fix works in real production traffic over a sustained period.

Duration: 24–48 hours of continuous monitoring

Steps:

1. **Monitoring**
   - Observe the system for the entire 24–48 hour window
   - Watch for unexpected side effects or regressions
   - Monitor relevant metrics (success rates, error rates, performance)
   - Keep logs organized and searchable

2. **Metric Comparison**
   - Run queries comparing "Before" vs. "After" metrics
   - Calculate percentage improvements (e.g., block rate reduction)
   - Analyze trends and patterns in the data
   - Document any anomalies or unexpected behavior

3. **Final Audit**
   - Check for regressions in other areas
   - Verify that the fix does not introduce new issues
   - Review edge cases and error scenarios
   - Confirm observability checkpoints are logging as expected

Deliverables:

- Performance summary with before/after metrics
- Comparative analysis (tables, graphs, trend data)
- Final GO/NO-GO decision with clear reasoning
- Documented evidence of fix effectiveness
- Any recommendations for future improvements

Phase C Completion Condition:

> "Phase [X]C verification complete. Final verdict: [GO/NO-GO with reasoning]."

Real-World Examples

Example 1: The Branding Bug (Phase 5)

Context: High block rate on video posts

Phase 5A (Investigation):
- Discovered that "banned fragments" (e.g., "But Comfortable") were not AI hallucinations
- Found hardcoded offending strings in `buildHookTitle()` templates
- Identified 6 templates with problematic fragments
- Evidence: Database queries showed 100% correlation between specific templates and blocks

Phase 5B (Implementation):
- Rewrote the 6 offending templates with alternative hooks
- Deduplicated code into `_shared/title-builder.ts` for maintainability
- Added logging to track template usage and block reasons
- Tested 50+ title generations with 100% pass rate

Phase 5C (Verification):
- Monitored for 48 hours post-deployment
- Block rate dropped from ~25–30% to <2%
- No regressions detected in other metrics
- Final verdict: **GO** - Fix highly effective

Example 2: Rendering Variety (Phase 6)

Context: Gainesville videos looked repetitive

Phase 6A (Investigation):
- Investigated why the rendering system was falling back to a repetitive style
- Discovered that "No Template" in Creatomate was valid for inline source
- Found a silent API failure was causing 100% fallback to JSON2Video
- Evidence: API error logs showed specific failure modes

Phase 6B (Implementation):
- Added structured logging to `_shared/video-render.ts`
- Captured specific API failure reasons (HTTP status codes and response bodies)
- Implemented retry logic with exponential backoff
- Added fallback selection logic with logging

Phase 6C (Verification):
- Monitored video rendering diversity metrics
- Confirmed variety increased by X%
- No increase in error rates or timeouts
- Final verdict: **GO** - Improvement sustained

Core Principles & Guidelines

1. Observability Before Action

- Always add logging/monitoring **before** changing behavior
- Make the effects of your changes visible and measurable
- Create clear audit trails for debugging and compliance

2. Standardized Logging

- All log events must include:
- `event_type`: What kind of event (e.g., “fix_checkpoint”, “error”, “success”)
- `context`: Relevant contextual data (city, slot, provider, user)
- `error_details`: HTTP status, error message, response body excerpt
- Use consistent formatting across all phases
- Ensure logs are searchable and aggregatable

3. Independent Revertability

- Design each phase so it can be rolled back independently
- Each phase should be independently reviewable
- Document explicit rollback procedures for each phase
- Ensure rollback does not break the entire system

4. Evidence-Driven Decision Making

- Base all decisions on data, not intuition
- Require multiple data points before forming conclusions
- Document assumptions and how they will be verified
- Record both supporting and contradicting evidence

5. Minimal, Surgical Changes

- Make only the changes necessary to fix the identified issue
- Avoid scope creep or opportunistic improvements
- Keep review surface area small and focused
- Preserve existing functionality

When to Use IFDP

The IFDP is most effective for:

- **Production outages or critical bugs** - Where understanding the root cause is worth the investigation time

- **Recurring or intermittent issues** - Where quick fixes have failed or caused regressions
- **Complex system failures** - Where multiple systems or services are involved
- **High-impact bugs** - Where the cost of a mistake is very high
- **Pattern-based issues** - Where you need to understand systemic problems, not just fix one instance

IFDP may be overkill for:

- Simple, obvious bugs with clear fixes
- Single-line typos or configuration errors
- Issues that have been fully understood and have known solutions
- Low-impact issues where fast deployment is more valuable than certainty

---

Workspace Integration

For GitHub/Version Control

- Store this protocol document in `/docs/IFDP_METHODOLOGY.md`
- Reference IFDP phase numbers in commit messages (e.g., “Phase 5A investigation: root cause identified”)
- Create pull requests for each phase (Phase A findings → Phase B implementation → Phase C verification)
- Tag verified fixes with the phase completion date

For AI Assistant Tools (Lovable, Cursor, etc.)

- Include a reference to this document in your workspace knowledge
- Reference IFDP phases when requesting assistance
- Ask the assistant to follow IFDP phases for complex debugging
- Use phase-specific prompts to maintain rigor and structure

For Team Communication

- Reference phase completion conditions in status updates
- Share phase findings before proceeding to implementation
- Document approval decisions and sign-offs
- Archive each phase’s findings for future reference

Checklist for Each Phase

Phase A Checklist

- [ ] Created flow diagram of affected system
- [ ] Executed relevant database/log queries
- [ ] Documented evidence with metrics
- [ ] Reviewed all related code sections
- [ ] Formed data-driven root cause hypothesis
- [ ] Identified 3-5 potential fix strategies

- [ ] Obtained user approval to proceed to Phase B

Phase B Checklist

- [ ] Implemented approved fix only (no scope creep)
- [ ] Added observability logging at key checkpoints
- [ ] Created or updated unit tests
- [ ] Achieved 100% test pass rate
- [ ] Verified clean deployment logs
- [ ] Confirmed rollback procedure is documented
- [ ] Obtained approval to proceed to Phase C

Phase C Checklist

- [ ] Monitored production for full 24–48 hour window
- [ ] Documented before/after metrics
- [ ] Analyzed comparison data and trends
- [ ] Checked for regressions in other areas
- [ ] Verified observability signals are present
- [ ] Made final GO/NO-GO decision
- [ ] Archived all findings and metrics

Frequently Asked Questions

Q: How long does IFDP typically take?
A: Investigation (Phase A) usually takes 2–4 hours. Implementation (Phase B) takes 2–8 hours depending on fix complexity. Verification (Phase C) spans 24–48 hours but doesn't require active work—mostly monitoring. Total time: 28–60 hours of calendar time, with 4–12 hours of active work.

Q: Can we skip Phase A if we think we know the problem?
A: No. Phase A is the most critical part. Many "quick fixes" fail because they address the symptom, not the root cause. Invest the time in Phase A to save time in the long run.

Q: What if Phase B breaks something?
A: That's why Phase C exists. If Phase C shows problems, you have a clear rollback path for Phase B only. The Phase A findings remain valid and can inform a revised Phase B approach.

Q: Who approves each phase?
A: This depends on your team structure. Typically: a senior engineer or tech lead reviews Phase A findings before proceeding to Phase B. A product or operations lead reviews Phase C verification before marking as "GO."

Q: Can phases overlap?
A: Not significantly. Phase A must be complete before Phase B begins. Phase B should be complete (and clean logs verified) before Phase C begins. However, Phase C monitoring can start immediately after Phase B deployment.

References

- See `IFDP_QUICK_REFERENCE.md` for a one-page summary
- See `LOVABLE_INSTRUCTIONS.md` for tool-specific guidance